

第六章 软件项目管理 AI讲解

开篇导读

一句话概括：软件项目管理是为了在规定时间内、预算内高质量完成软件项目，核心是进度管理、风险管理、质量保证、配置管理和过程成熟度评估。

本章在考试中的地位：中高频考点章。关键路径计算、CMM五级特征、风险应对策略是必考点。本章坑点在"关键路径=最长路径（不是最短）"、CMM等级的反向设问、PERT公式的计算。

注：本章面向对象基础仅作背景铺垫，不作为主要考点。

一、进度管理

核心概念

进度管理的目的是合理安排任务，确保项目按时完成。主要工具：

1. 甘特图（Gantt图）：用条形图直观显示各任务的开始/结束时间和进度
2. PERT图/CPM（关键路径法）：用网络图分析任务依赖关系，计算工期

关键路径 必考

关键路径是项目网络图中耗时最长的路径，它决定了项目的最短工期。

为什么重要

这是最高频的坑点！"关键路径"听名字像"最短路径"，但实际上是最长路径——因为项目必须等所有任务完成，所以工期由最慢的那条路径决定。

关键路径的特点

- 是所有路径中耗时最长的
- 决定项目的最短工期（不是最短路径！）
- 关键路径上的任务叫关键任务，不能延误（延误则整个项目延误）
- 关键路径上的松弛时间为0（没有缓冲时间）

完整算例：如何找关键路径

项目网络图（5个任务，标注工期和依赖）：

任务A(3天) → 任务B(4天) → 任务D(2天) → 任务E(3天)
任务A(3天) → 任务C(5天) → 任务E(3天)

找所有路径并计算总工期：

路径	计算	总工期
A→B→D→E	3+4+2+3	12天
A→C→E	3+5+3	11天

最长路径是 A→B→D→E（12天），这就是关键路径，项目最短工期=12天。

恍然大悟：为什么关键路径是最长而不是最短？因为项目要等所有任务完成才能结束——就像一群人



爬山到山顶汇合，到达时间取决于最慢的那个人，最慢的路径就是"关键"的，它决定整体进度。路径A→C→E虽然只要11天，但它不是关键路径，因为它比A→B→D→E快，能提前完成不影响整体。

ES/EF/LS/LF和松弛时间计算（计算题核心）

四个关键参数：

- ES (Earliest Start)：最早开始时间
- EF (Earliest Finish)：最早结束时间 = ES + 工期
- LS (Latest Start)：最晚开始时间（不延误项目的最晚开始）
- LF (Latest Finish)：最晚结束时间 = LS + 工期
- 松弛时间 = LS - ES = LF - EF（可以拖延的时间）

计算方法：

- 正向推算ES/EF：从起点往后推，取所有前置任务EF的最大值作为当前ES
- 逆向推算LS/LF：从终点往前推，取所有后续任务LS的最小值作为当前LF

用上面的例子计算（项目总工期12天）：

任务	工期	ES	EF	LS	LF	松弛	是否关键
A	3	0	3	0	3	0	✓关键
B	4	3	7	3	7	0	✓关键
C	5	3	8	4	9	1	非关键
D	2	7	9	7	9	0	✓关键
E	3	9	12	9	12	0	✓关键

解读：A/B/D/E松弛时间=0，是关键任务；C松弛时间=1，可以拖延1天不影响项目。

恍然大悟：松弛时间为什么能判断关键任务？因为松弛时间="可以拖延的时间"——松弛为0就是"一分钟都不能拖"，这种任务一旦延误整个项目就延误，所以它才是"关键"的。C有1天松弛，说明C慢1天项目还能按时完成，所以C不是关键的。

PERT计算 计算题

PERT（计划评审技术）用三种时间估计计算期望时间：

期望时间 = (乐观时间 + 4×最可能时间 + 悲观时间) / 6

标准差 = (悲观时间 - 乐观时间) / 6

PERT完整算例

题目：某任务乐观估计4天，最可能6天，悲观14天，求期望时间和标准差。

计算：

- 期望时间 = $(4 + 4 \times 6 + 14) / 6 = (4 + 24 + 14) / 6 = 42 / 6 = 7$ 天
- 标准差 = $(14 - 4) / 6 = 10 / 6 \approx 1.67$ 天

解读：

- 期望时间7天：该任务的平均工期约7天
- 标准差1.67天：工期的不确定性约±1.67天（标准差越大，不确定性越高）

恍然大悟：PERT为什么用三个时间而不是一个？因为研发类任务不确定性高——你不知道实际要4天还是14天，只给一个数太冒险。PERT用"乐观+4×最可能+悲观"加权平均，最可能时间权重最大（×4），乐观和悲观各占1份，这样既考虑了最可能的情况，又兼顾了极端情况。标准差则量化"有多不确定"——标准差大说明乐观悲观差距大，风险高。

易混辨析

考法	易错点
----	-----

"关键路径是最长还是最短"	是最长路径！决定最短工期。不是最短路径
"关键路径上的任务有什么特点"	松弛时间为0，不能延误
"PERT期望时间公式"	$(\text{乐观} + 4 \times \text{最可能} + \text{悲观}) / 6$ ，不是简单平均
"PERT标准差公式"	$(\text{悲观} - \text{乐观}) / 6$

记忆技巧

口诀："关键最长定工期"——关键路径是最长的，决定工期。

PERT口诀："乐四可悲除以六"（乐观+4×最可能+悲观，除以6）。

标准差口诀："悲减乐除以六"（悲观-乐观，除以6）。

二、风险管理

核心概念

风险管理是识别、评估、应对项目风险的过程。

为什么重要

软件项目充满不确定性——技术可能不成熟、需求可能变化、人员可能流失、进度可能延误。如果不提前管理风险，等到问题爆发时就为时已晚。风险管理的本质是"防患于未然"——提前识别风险、评估影响、制定对策，把"突发事件"变成"可控事件"。

类比：风险管理就像买保险+备应急预案——你不知道事故会不会来，但提前准备好了应对方案。

风险管理四步骤

1. 风险识别：找出可能的风险
2. 风险估计：评估风险的概率和影响
3. 风险驾驭：制定应对策略
4. 风险监控：持续跟踪风险状态

四步骤深度拆解

1. 风险识别 (Risk Identification)

- 是什么：尽可能列出项目中可能发生的风险
- 常见风险类型：
 - 技术风险：技术方案不可行、新技术不成熟
 - 进度风险：估算偏差、资源延迟、需求变更
 - 人员风险：核心人员离职、团队经验不足
 - 需求风险：需求蔓延 (Scope Creep)、需求不清
 - 外部依赖风险：第三方服务不稳定、合作方延期
- 识别方法：
 - 检查表 (Checklist)：用历史项目积累的风险清单逐条对照
 - 头脑风暴：团队集体讨论可能风险
 - 专家访谈：请有经验的人评估风险
 - 历史项目分析：从过往类似项目找模式
- 输出：风险清单 (每条含编号、描述、类型)



2. 风险估计 (Risk Estimation)

- 是什么：评估每个风险的发生概率和影响严重性
- 核心公式：风险暴露度 $RE = P \times L$ (Risk Exposure = Probability $\times$ Loss)
- 量化方法：
 - 概率分级：高 (>70%) / 中 (30-70%) / 低 (<30%)
 - 影响分级：高 (项目延期>30%或损失>30%) / 中 / 低
- 二维矩阵：3×3矩阵区分9个风险等级
- 示例：核心人员离职 $P=20\%$ $L=$ 进度延期2个月 $\rightarrow RE=0.2 \times 60\text{天}=12\text{天}$ 等价进度损失
- 输出：带风险等级的风险清单 (按RE排序)

3. 风险驾驭 (Risk Mitigation)

- 是什么：对每个高风险制定具体应对计划——RMMM (Risk Mitigation, Monitoring, Management)
- 三个层次：
 - Mitigation (缓解)：减少风险发生概率或影响 (如做技术原型验证)
 - Monitoring (监测)：定期检查风险是否触发 (如每周风险评审会)
 - Management (管理)：风险触发后的应急计划 (如核心人员离职后的knowledge transfer计划)
- 驾驭策略：选择"避/转/减/接"四种策略之一 (详见下节)
- 输出：每个高风险的RMMM计划

4. 风险监控 (Risk Monitoring)

- 是什么：持续跟踪风险状态，及时发现风险触发或新风险
- 监控手段：
 - 风险跟踪表：每周更新风险状态 (已发生/部分发生/已解除)
 - 定期评审：每周/双周风险评审会
 - 触发条件预警：设定阈值 (如进度落后10% $\rightarrow$ 触发风险预警)
 - 新风险纳入：发现新风险则回到步骤1重新走流程

四步骤的PDCA循环

恍然大悟：风险四步类似PDCA循环精神——识别 (提前规划) $\rightarrow$ 估计 (评估优先级) $\rightarrow$ 驾驭 (制定应对) $\rightarrow$ 监控 (持续跟踪并回到识别)。这不是一次性活动，而是周而复始的闭环——每次评审会都重新走4步：有新风险吗？已识别风险等级变了吗？驾驭计划生效了吗？需要调整吗？风险管理的核心不是"消除风险" (不可能)，而是"持续掌控风险"——这是大型项目能够交付的关键能力。

风险应对策略 必考

策略	含义	通俗解释
规避	改变计划，避开风险	"绕道走"
转移	把风险转给第三方 (如买保险)	"甩锅"
减轻	降低风险的概率或影响	"减小伤害"
接受	接受风险，不采取行动	"认了"

易混辨析

考法	易错点
"买保险属于什么策略"	转移 (把风险转给保险公司)
"改变技术方案避免风险"	规避 (绕开风险)
"增加测试降低缺陷率"	减轻 (降低概率/影响)
"知道有风险但不处理"	接受 (认了)



记忆技巧

口诀：“避转减接”（规避、转移、减轻、接受）。

场景判断：

- 买保险→转移
- 换方案→规避
- 加测试→减轻
- 不处理→接受

三、软件质量保证（SQA）

核心概念

软件质量保证（SQA）是有计划、系统地确保软件质量的一系列活动。

主要方法

1. 技术审查：正式的技术评审
2. 程序正确性证明：用数学方法证明程序正确（理论性强，实践少）
3. 走查（Walkthrough）：非正式的小组审查，由设计者带领大家“走”一遍
4. 审查（Inspection）：正式的缺陷检测流程，有严格步骤和角色分工

SQA深度拆解

- 是什么：贯穿软件生命周期的、独立于开发团队的质量监督活动
- 核心特征：
 1. 独立性——SQA组通常独立于开发团队，向高层汇报，避免“裁判员同时是运动员”
 2. 全周期性——从需求阶段到维护阶段都有SQA活动
 3. 流程驱动——按既定流程执行，不依赖个人判断
 - SQA组的职责：
 - 制定质量计划（SQAP，Software Quality Assurance Plan）
 - 评审各阶段产出物（SRS/设计文档/代码/测试用例）
 - 维护缺陷跟踪系统
 - 审计开发团队是否遵守流程
 - SQA活动贯穿全生命周期：

阶段	SQA活动
需求	需求评审、SRS质量检查
设计	设计评审、架构审查
编码	代码审查、单元测试评审
测试	测试计划评审、测试报告审计
维护	变更控制审计、缺陷率统计

四种主要方法详细对比

方法	性质	主导者	适用阶段	严格程度
技术审查	正式	评审组长	各阶段产出物评审	中



程序正确性证明	形式化	数学家/形式化专家	关键算法/安全代码	最高
走查 (Walkthrough)	非正式	设计者本人	设计/代码早期	低
审查 (Inspection)	正式	主持人 (非作者)	设计/代码评审	高

程序正确性证明（了解）

- 是什么：用数学方法（如Hoare逻辑、模型检查）证明程序逻辑正确
- 典型方法：前置条件{P} 程序代码{S} 后置条件{Q}，证明 {P} S {Q}
- 优点：100%正确性保证（数学证明）
- 缺点：成本极高、规模受限（只能证小规模代码）、需专业知识
- 实际应用：航天、核电、医疗等关键安全系统的核心算法

走查 vs 审查（深度对比）

维度	走查 (Walkthrough)	审查 (Inspection)
正式程度	非正式	正式
主持人	设计者本人	第三方主持人（非作者）
目的	解释设计、收集反馈	系统性查缺陷
流程	简单（开会讨论）	严格（6步Fagan审查）
参与角色	同事自由参与	明确分工（主持人/作者/审查员/记录员）
输出	改进建议（口头）	缺陷清单+评估报告
类比	"同事互看作业"（友好讨论）	"项目审计"（严格找错）

Fagan审查的6步标准流程

由Michael Fagan在IBM 1976年提出，被视为最有效的代码评审方法：

1. 计划 (Planning)：选定审查对象、组建审查小组
2. 概览 (Overview)：作者向审查员介绍背景和设计意图
3. 准备 (Preparation)：审查员独立阅读材料、记录可疑点（每人2-4小时）
4. 审查 (Inspection Meeting)：召开会议逐项讨论缺陷
5. 返工 (Rework)：作者修复缺陷
6. 跟踪 (Follow-up)：主持人验证修复，决定是否需重审

恍然大悟：走查像"同事互看作业"——气氛友好但效率低；审查像"项目审计"——气氛严肃但找bug多。研究数据表明，正式审查能找出50-80%的缺陷，远高于走查的20-30%。正式程度差异决定了找bug效率——所以关键模块的代码评审必须用审查（Fagan流程），日常迭代代码可用走查或Pull Request评审。SQA组的核心价值是"独立性"——开发自己评审自己的代码不算SQA（裁判员=运动员），必须有第三方主持。这就是为什么大公司都设独立的QA团队向VP汇报，而非汇报给开发经理。

记忆技巧

口诀："技正走审"（技术审查、正确性证明、走查、审查）。走查=非正式"走一遍"，审查=正式"查缺陷"。

Fagan六步："计概准审返跟"（计划→概览→准备→审查→返工→跟踪）。

四、软件配置管理（SCM）



核心概念

软件配置管理（SCM）是管理软件生命周期中的变更，确保软件产品的完整性和可追溯性。

为什么重要

软件在开发过程中会不断变化——需求变更、bug修复、新功能添加。如果没有配置管理，就会出现"谁改了什么？""现在用的是哪个版本？""改了之后出了问题怎么回退？"等混乱局面。配置管理就像给软件的每个状态"拍照存档"，让任何时刻都能追溯和回退。

类比：配置管理就像Git版本管理——每次提交都是一个快照，可以随时回退、对比、分支开发。

五大核心概念

1. 配置项：需要管理的工件（源代码、文档、测试用例等）
2. 基线：经过正式评审、批准的配置项快照，作为后续变更的基准
3. 版本控制：管理配置项的不同版本
4. 变更控制：控制对基线的修改
5. 配置审计：验证配置项是否符合要求

五大概念深度拆解

1. 配置项（Configuration Item, CI）

- 是什么：纳入配置管理范围的所有工件
- 典型配置项：
 - 代码类：源代码、配置文件、构建脚本
 - 文档类：需求规格说明书、设计文档、用户手册
 - 测试类：测试用例、测试数据、测试报告
 - 环境类：开发/测试/生产的环境配置
- 命名规则：通常用"工件类型_模块_版本"（如SRS_订单模块_v1.2）
- 判断标准：是否需要追踪历史变更？需要→纳入配置项

2. 基线（Baseline）

- 是什么：经过正式评审和批准的配置项快照，作为后续工作的参考点和变更基准
- 三类基线（对应不同生命周期阶段）：

基线类型	建立时机	包含内容
功能基线	需求评审通过后	SRS、需求矩阵
分配基线	设计评审通过后	概要设计文档、接口规约
产品基线	系统测试通过后	最终代码、用户文档、安装包

- 核心特性：基线是冻结的——一旦确立，修改必须走变更控制流程，不能随意改动
- 作用：保护已经达成共识的成果不被随意推翻

3. 版本控制（Version Control）

- 是什么：管理配置项的多个版本，支持对比、回退、分支
- 核心机制：
 - 版本号规则：通常为主版本.次版本.修订版本（如v1.2.3）
 - 分支策略：主干（main）+ 开发分支（develop）+ 特性分支（feature/x）+ 修复分支（hotfix/y）
 - 合并冲突：多人改同一文件时需手动解决冲突
- 典型工具：Git、SVN、Perforce

- 与基线的关系：基线是正式冻结的版本，普通版本不一定冻结

4. 变更控制 (Change Control)

- 是什么：对基线进行修改时的正式审批流程
- CCB (Change Control Board, 变更控制委员会)：决策机构，由项目经理、技术负责人、QA 代表、客户代表组成
- 标准变更流程 (5步)：

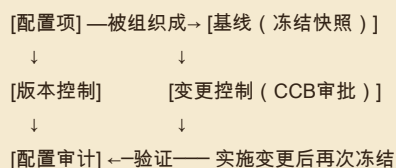
1. 提交变更请求 (CR)：写明改什么、为什么、影响范围
2. 评估：技术评估 (成本/工作量) + 业务评估 (必要性/优先级)
3. CCB审批：批准/拒绝/延期
4. 实施：按批准方案修改代码/文档

5. 验证：测试验证修改正确，更新基线
 - 目的：避免"任何人随意改基线"导致的混乱

5. 配置审计 (Configuration Audit)

- 是什么：验证配置项是否真实满足要求的检查
- 两类审计：
 - 功能配置审计 (FCA)：验证产品功能满足需求文档 ("做的对吗")
 - 物理配置审计 (PCA)：验证产品组成完整 ("该有的都有吗"——文档、源码、安装包是否齐全)
- 执行时机：每次基线建立前必做、产品发布前必做
- 输出：审计报告 (合规/不合规清单)

五大概念的协作关系



易混辨析

考法	易错点
"基线vs版本"	基线是正式冻结的版本 (经评审批准, 改它要走变更流程) ; 普通版本不一定冻结, 可随意修改
"版本控制vs变更控制"	版本控制管"历史记录" (每个版本都存着) ; 变更控制管"修改审批" (改基线要打报告)
"三类基线顺序"	功能基线 (需求阶段) → 分配基线 (设计阶段) → 产品基线 (测试通过后)
"FCA vs PCA"	FCA验证功能 (做的对吗) ; PCA验证物理组成 (该有的都有吗)

恍然大悟：SCM是软件版本的"户口管理"——每个工件都有身份 (配置项)，关键时刻拍照 (基线)，变动要审批 (变更控制)，定期点名核对 (配置审计)。基线是SCM的"骨架"，没有基线就没有"已确立的事实"，所有协作都是空谈。变更控制是SCM的"法律"，没有它任何人都能随意改代码改需求，团队就乱套。SCM不是为了"方便"，而是为了"可追溯"——出了问题能查到"谁在什么时候改了什么"，能回退到任何历史版本。这是软件工程从手工作坊走向工业化的关键能力。Git的成功证明了好的SC

M工具能改变整个行业——但工具只是手段，配置管理的核心是流程和纪律。

记忆技巧

口诀："项基版变审"（配置项、基线、版本控制、变更控制、配置审计）。

基线口诀："功分产"——功能基线（需求）、分配基线（设计）、产品基线（测试通过）。

变更5步："提评批改验"——提交→评估→批准→实施（修改）→验证。

五、CMM/CMMI 五级 必考

核心概念

CMM（软件过程能力成熟度模型）用于评估软件组织的开发过程成熟度，分5级（以张海藩教材为准，采用CMM叫法）。

为什么重要

CMM回答的是一个核心问题："这个组织开发软件的过程靠不靠谱？"。1级组织靠"个人英雄主义"——能不能做好全看运气和个人能力；5级组织靠"制度+数据+持续改进"——不管谁来都能稳定产出高质量软件。CMM就像企业成熟度评级，等级越高，过程越规范、越可预测。

等级	名称	特征
1级	初始级	过程不可预测，靠个人英雄主义
2级	可重复级	有项目级纪律，能重复成功经验
3级	已定义级	有组织级标准过程
4级	已管理级	用定量指标度量过程（注：CMMI叫"量化管理级"）
5级	优化级	持续改进过程

教材差异提示：CMM第4级张海藩版叫"已管理级"，CMMI叫"量化管理级"。期末考试以张海藩版"已管理级"为准。

五级关键过程域（KPA）深度拆解

每级有专属的关键过程域（Key Process Areas，KPA），是该级必须做到的核心实践：

1级 → 2级（最难的跃迁：建立纪律）

2级 KPA（6个核心，记2-3个即可）：

- 需求管理——管好需求变更
- 项目计划——制定可执行的项目计划
- 项目跟踪——跟踪项目进度与计划偏差
- 配置管理——本章第四节SCM内容
- 质量保证——本章第三节SQA内容
- 分包管理——管理外包/采购

核心特征：项目级纪律——单个项目能成功且经验可复用

2级 → 3级（建立标准）

3级 KPA（关键3个）：

- 组织过程定义——定义组织级标准过程（不再各项目各搞一套）
- 培训计划——系统化培训新员工

- 集成软件管理——把组织标准过程裁剪到具体项目
- 注：3级在CMM v1.1中共定义7个KPA，本节只列出最常考3个，其他4个为：同行评审（对应本章第三节走查/审查）、软件产品工程、组间协调、组织过程焦点。考试若涉及"同行评审属于第几级"答案是3级。

核心特征：从"项目级"升级到"组织级"——所有项目按统一标准做

3级 → 4级（数据驱动）

- 4级 KPA（核心2个）：
- 定量过程管理——用统计方法度量过程能力（如缺陷率、生产率）
 - 软件质量管理——基于定量数据管理质量目标

核心特征：用数据说话——能预测项目结果（不再凭感觉）

4级 → 5级（持续优化）

- 5级 KPA（核心3个）：
- 缺陷预防——分析缺陷根因，预防再发生（不只是修复）
 - 技术变更管理——系统化引入新技术
 - 过程变更管理——持续改进过程本身

核心特征：自我进化——过程在不断变得更好

等级跃迁的难度

跃迁	难度	关键挑战
1→2	最难（过滤掉70%组织）	建立基本纪律，从混乱到有序
2→3	难	从单项目纪律升级到组织级标准
3→4	中	需要数据基础（没有度量数据想4级也做不到）
4→5	较难	需要文化变革，从"完成任务"到"持续改进"

恍然大悟：CMM五级是"从靠人到靠制度再到靠数据再到靠文化"的演进——

- 1级靠英雄（项目成败看运气）
- 2级靠流程（项目级纪律，能复用经验）
- 3级靠标准（组织级标准过程）
- 4级靠数据（用度量预测结果）
- 5级靠改进文化（持续优化过程本身）

>

这条演进路径的本质是"对项目结果的可预测性"逐级提升——1级完全不可预测，5级高度可预测。1→2是最难的跃迁，因为要从"个人英雄主义"过渡到"流程纪律"，文化阻力最大。中国大部分软件公司停留在2-3级，少数大厂能达到4-5级。CMM不是"等级越高越好"——根据业务性质选择合适的等级即可（创业公司2级足够，航天/金融需要4-5级）。

易混辨析（反向设问高频）

考法	易错点
"能重复成功经验"是第几级	2级可重复级（关键词"重复"）
"有组织标准过程"是第几级	3级已定义级（关键词"定义/标准"）
"定量度量"是第几级	4级已管理级（关键词"定量/管理"）
"持续改进"是第几级	5级优化级（关键词"优化/改进"）

记忆技巧

五级口诀：“初重定管优”（初始、可重复、已定义、已管理、优化）。

特征关键词对应：

- 初始：不可预测、混乱
- 可重复：重复成功、项目纪律
- 已定义：标准过程、组织级
- 已管理：定量度量、数据驱动
- 优化：持续改进

六、面向对象基础（背景铺垫，非主要考点）

核心概念

面向对象的基本概念：

- 类：对象的抽象模板
- 对象：类的实例
- 封装：隐藏内部细节，暴露接口
- 继承：子类继承父类的属性和方法
- 多态：同一消息不同对象有不同响应
- 消息：对象间的通信

UML图分类

- 静态图：类图、对象图、用例图、组件图、部署图
 - 动态图：顺序图、协作图、状态图、活动图
- 本章以项目管理为主，面向对象仅作背景了解。

考点聚焦

高频考点清单

1. 关键路径=最长路径（决定最短工期）
2. CMM五级名称和特征（反向设问）
3. PERT期望时间公式 $(\text{乐观} + 4 \times \text{最可能} + \text{悲观}) / 6$
4. 风险应对四策略（避转减接）
5. PERT标准差公式 $(\text{悲观} - \text{乐观}) / 6$
6. 走查vs审查
7. 配置管理核心概念（基线、版本控制）
8. 面向对象基本概念（背景）

常见题型

- 计算题：PERT期望时间、标准差
- 判断题：关键路径是最长还是最短
- 反向设问：给出CMM特征判断等级



记忆口诀总览

1. 关键路径："关键最长定工期"
 2. PERT期望："乐四可悲除以六"
 3. PERT标准差："悲减乐除以六"
 4. 风险策略："避转减接"
 5. 质量方法："技正走审"
 6. 配置管理："项基版变审"
 7. CMM五级："初重定管优"
-

本章小结

本章核心逻辑链：进度管理（关键路径定工期）→ 风险管理（避转减接）→ 质量保证（技正走审）→ 配置管理（项基版变审）→ 过程成熟度（CMM五级）。

本章最大的坑点是关键路径的方向——它是最长路径（决定最短工期），不是最短路径！很多同学看到"关键"就以为是最短/最优，实际上关键路径是最慢的那条路。其次是CMM等级的反向设问——给出特征判断等级，需要把五级的特征关键词记牢。

掌握关键路径计算、PERT公式、CMM五级特征、风险四策略，本章就能稳稳拿分。